

PlanQuest Developer Execution Plan

Detailed implementation sequence for upgrading the current project into a production - style collaborative planner portfolio app.

Prepared: 2026-05-29
Audience: solo developer / future collaborators
Source of truth: docs/superpowers/plans/2026-05-29-planquest-master-execution-plan.md

Phase	Primary Outcome	Workload	Exit Signal
1	Lock product direction	Low code / high clarity	MVP boundaries approved
2	Refactor frontend architecture	Heavy refactor	Domain-based UI structure
3	Harden collaboration backend	Heavy backend	Two-user rules enforced
4	Add quality workflow	Tooling focused	CI + lint + test gates live
5	Ship event collaboration slice	Full-stack delivery	Two-user flow demo-ready
6	Rework challenges	Feature normalization	Challenge logic coherent
7	Prepare deployment	Ops and release	Portfolio-grade release

PlanQuest Master Execution Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Turn the current PlanQuest codebase from a promising intermediate portfolio app into a production-style collaborative planner platform with clear product scope, maintainable frontend/backend architecture, automated quality checks, and a polished deployable vertical slice.

Architecture: The implementation should proceed in layered phases. We first lock the product direction, then refactor the codebase so collaboration features have stable boundaries, then harden engineering workflow, and only then expand product behavior. The backend remains Django + DRF with domain apps; the frontend remains Next.js App Router, but the current single large client experience is decomposed into route-level screens, domain components, and API/session utilities.

Tech Stack: Django 5, Django REST Framework, SimpleJWT, Next.js 15 App Router, React 19, SQLite for local development moving toward PostgreSQL readiness, local media during development, CI automation, linting, and staged deployment.

Plan Scope

This is a master execution plan for the whole project. It is intentionally more operational than a normal roadmap and more architectural than a single-feature task list. Each phase below is meant to create a stable checkpoint that can be implemented, verified, and reviewed before the next phase begins.

The plan assumes the current baseline:

- Backend API exists and current tests pass.
- Frontend builds successfully but is structurally concentrated in one large component.
- Collaboration is part of the product idea, but much of the implemented logic is still creator-centric rather than multi-user.
- Engineering workflow is not yet production-style because linting, CI, deployment, observability, and environment discipline are incomplete.

Current Codebase Assessment

Strengths

- Backend has meaningful domain separation in backend/apps/accounts, backend/apps/planner, backend/apps/challenges, and backend/apps/achievements.
- Authentication is real and already wired through custom user model + JWT.
- There is an initial automated backend test suite covering auth, planner, challenges, and achievement logic.
- Frontend is already connected to the backend and demonstrates end-to-end user interaction.

Structural Risks

- The main frontend user experience lives mostly inside `frontend/components/auth/AuthExperience.jsx`, which is too large to scale comfortably.
- Collaboration rules are not yet first-class. The product promise says "collaborative/social," but the implementation still behaves mainly like a single-user app with adjacent social concepts.
- Backend business logic is still close to serializers/viewsets rather than clearly separated into application services and policy layers.
- No established CI, lint, release workflow, staging path, or production configuration baseline exists yet.

Definition Of Success For This Plan

- The project has a locked product direction and a controlled feature scope.
- The frontend is split into maintainable domain-based modules and route flows.
- The backend expresses collaboration rules explicitly through permissions, services, and tests.
- Quality gates run automatically on every push.
- At least one collaboration-heavy vertical slice feels production-like, not demo-like.
- A staging-ready deployment path exists and the portfolio version can be demonstrated confidently.

File Structure Strategy

This phase plan expects us to preserve the current repo split while growing clearer boundaries.

Existing Areas We Will Keep

- `backend/config/`
- `backend/apps/accounts/`
- `backend/apps/planner/`
- `backend/apps/challenges/`
- `backend/apps/achievements/`
- `frontend/app/`
- `frontend/components/`
- `frontend/lib/`
- `docs/`

New Or Expanded Areas We Should Introduce

- `frontend/app/(auth)/`
- `frontend/app/(dashboard)/`
- `frontend/components/events/`
- `frontend/components/challenges/`
- `frontend/components/profile/`
- `frontend/components/achievements/`
- `frontend/hooks/`
- `frontend/lib/session/`

- frontend/lib/validators/
- backend/apps/planner/services.py
- backend/apps/planner/permissions.py
- backend/apps/challenges/permissions.py
- backend/apps/common/ if shared utilities start repeating
- backend/tests/ or more structured per-app test modules if current tests grow too large
- .github/workflows/
- docs/architecture/
- docs/release/

Phase 1: Product Alignment And Delivery Strategy

Objective: Remove ambiguity from the product before we refactor or add major new features.

Why this phase is first: If we keep building without deciding whether PlanQuest is planner-first, challenge-first, or a carefully-scoped hybrid, the architecture will keep absorbing conflicting behavior.

Developer workload focus: Mostly documentation, product thinking, and system scoping, but this phase prevents expensive rework later.

Required Outputs

- Product direction memo
- MVP scope document
- V1 backlog
- Non-goals list
- Primary user journey map
- Data ownership and visibility rules document

Files

- Create docs/architecture/planquest-product-direction.md
- Create docs/architecture/planquest-user-journeys.md
- Create docs/architecture/planquest-domain-rules.md
- Update README.md
- Update docs/project-brief.md

Developer Steps

- Audit the current product language in README.md and docs/project-brief.md.
- List every currently implemented capability and mark each as core, supporting, or unclear.
- Write one sentence answering: "What is the primary reason a user returns to PlanQuest every week?"
- Decide the product posture:
 - planner-first with social accountability
 - challenge-first with scheduling support

- balanced hybrid, only if the journeys are truly cohesive
- Define one primary persona and one secondary persona.
- Write 3 canonical journeys:
 - register and set up account
 - create and manage event collaboration
 - create and complete challenge with visible progress
- Define visibility rules:
 - who can see events
 - who can invite whom
 - who can log challenge progress
 - who can complete group challenges
 - who can see achievements
- Freeze all stretch ideas that do not support the first polished collaboration slice.

Verification

- The product can be described in under 60 seconds without contradicting itself.
- Every existing or proposed feature maps to at least one primary journey.
- At least one important feature is deliberately postponed to protect focus.

Exit Criteria

- Product direction is explicit.
- MVP and V1 are separated.
- The next architecture phase has stable product assumptions to build on.

Phase 2: Frontend Architecture Reset

Objective: Replace the current single large UI container with domain-based frontend structure that can support real feature growth.

Why now: The frontend currently works, but the current structure will slow every future feature, make bugs harder to isolate, and turn UI state into a maintenance problem.

Developer workload focus: Refactoring, route design, state boundary cleanup, and domain decomposition.

Required Outputs

- Route-level page structure
- Session/bootstrap abstraction
- Domain components for auth, events, challenges, profile, achievements
- Shared API error/loading conventions

- Frontend linting baseline

Primary Files To Modify

- Modify frontend/app/page.jsx
- Modify frontend/app/layout.jsx
- Modify frontend/app/globals.css
- Refactor frontend/components/auth/AuthExperience.jsx
- Modify frontend/lib/api.js

Files To Create

- Create frontend/app/(auth)/login/page.jsx
- Create frontend/app/(auth)/register/page.jsx
- Create frontend/app/(dashboard)/dashboard/page.jsx
- Create frontend/app/(dashboard)/events/page.jsx
- Create frontend/app/(dashboard)/challenges/page.jsx
- Create frontend/app/(dashboard)/profile/page.jsx
- Create frontend/components/auth/LoginForm.jsx
- Create frontend/components/auth/RegisterForm.jsx
- Create frontend/components/layout/Sidebar.jsx
- Create frontend/components/layout/Topbar.jsx
- Create frontend/components/events/EventList.jsx
- Create frontend/components/events/EventModal.jsx
- Create frontend/components/events/DeleteEventDialog.jsx
- Create frontend/components/challenges/ChallengeList.jsx
- Create frontend/components/challenges/ChallengeCard.jsx
- Create frontend/components/challenges/ProgressForm.jsx
- Create frontend/components/achievements/AchievementShelf.jsx
- Create frontend/components/profile/ProfileSummary.jsx
- Create frontend/hooks/useSessionBootstrap.js
- Create frontend/hooks/useAuthSession.js
- Create frontend/lib/session/token-storage.js
- Create frontend/lib/errors.js

Developer Steps

- Identify all state currently owned by AuthExperience.jsx.
- Group that state into domains:
 - auth/session
 - events
 - challenges
 - achievements

- UI overlays/modals

- Move session token storage and retrieval into frontend/lib/session/token-storage.js.
- Move initial load orchestration into frontend/hooks/useSessionBootstrap.js.
- Split auth form rendering into dedicated components.
- Split dashboard shell rendering from domain panels.
- Replace one-page anchor navigation with route-based screens or nested dashboard sections backed by pages.
- Standardize how loading, empty, and error states appear across all feature screens.
- Reduce prop drilling by moving related state closer to domain roots.
- Keep styling consistent with the existing visual language unless intentionally redesigning later.

Recommended Refactor Sequence

- Extract pure utility functions from AuthExperience.jsx first.
- Extract token/session behavior second.
- Extract auth forms third.
- Extract dashboard shell fourth.
- Extract event domain components fifth.
- Extract challenge and achievement components sixth.
- Only after the code is decomposed, introduce route-level page segmentation.

Verification

- Frontend production build still passes after each major extraction checkpoint.
- No behavior regressions in login, register, create event, edit event, delete event, create challenge, log progress, and complete challenge.
- New contributors can find the event UI without reading auth code and vice versa.

Exit Criteria

- No single frontend component remains the owner of the whole app experience.
- Session flow is reusable and testable.
- UI work can proceed by domain instead of by giant file surgery.

Phase 3: Backend Collaboration Model Hardening

Objective: Make backend behavior match the collaborative product promise.

Why now: The current backend is solid for a personal MVP, but the collaboration logic is not yet explicit enough for a portfolio app presented as multi-user software.

Developer workload focus: Data model evolution, permission rules, serializer adjustments, business services, and test expansion.

Required Outputs

- Explicit collaboration rules for events

- Invitation lifecycle
- Group challenge design decision
- Cleaner service boundaries
- Expanded tests around ownership and visibility

Primary Files To Modify

- Modify backend/apps/planner/models.py
- Modify backend/apps/planner/serializers.py
- Modify backend/apps/planner/views.py
- Modify backend/apps/challenges/models.py
- Modify backend/apps/challenges/serializers.py
- Modify backend/apps/challenges/views.py
- Modify backend/apps/achievements/services.py
- Modify backend/config/urls.py

Files To Create

- Create backend/apps/planner/services.py
- Create backend/apps/planner/permissions.py
- Create backend/apps/challenges/services.py
- Create backend/apps/challenges/permissions.py
- Create backend/apps/planner/tests/test_event_visibility.py
- Create backend/apps/planner/tests/test_event_invitations.py
- Create backend/apps/challenges/tests/test_group_challenges.py
- Create backend/apps/challenges/tests/test_progress_permissions.py

Developer Steps

- Decide whether event creator is automatically an accepted participant or remains separate from invitees.
- Define event visibility states:
 - creator-only event
 - invited user pending
 - invited user accepted
 - invited user declined
- Extend event query logic so users can retrieve events they own and events they are participating in, according to status rules.
- Add invitation creation/update flows that prevent duplicate participant rows and unauthorized edits.
- Move event invitation logic out of viewsets into planner services where mutation rules are easier to test.
- Decide the first implementation path for challenges:

- keep self-challenges only for MVP slice, or
- add minimal group challenge support with explicit constraints
- If group challenges are added, define who can add participants, who can log progress, and how completion is resolved.
- Revisit achievement awarding logic so it uses domain events/services rather than ad hoc calls inside viewsets.
- Review serializer exposure and remove any accidental fields that should be computed or hidden.

Data And Migration Concerns

- Audit current migrations before adding collaboration changes.
- Keep migration steps additive when possible.
- If semantics change significantly, write migration notes in docs/architecture/planquest-domain-rules.md.

Verification

- A creator cannot edit another user's invitation state unless the policy explicitly allows it.
- An invited user can only view or mutate event data according to participation rules.
- Challenge progress rules behave consistently across self and group flows.
- Achievement awarding remains idempotent.
- Backend tests remain green and new collaboration tests cover the main branch conditions.

Exit Criteria

- Collaboration exists as an intentional system, not a future idea.
- The backend can support a real two-user demo without hand-waving around ownership.

Phase 4: Engineering Quality Foundation

Objective: Introduce the workflow discipline expected in real software teams.

Why now: Once architecture is steadier, automation and quality gates begin protecting the project instead of blocking early experimentation.

Developer workload focus: Tooling, linting, formatting, automated checks, and team operating rules.

Required Outputs

- Frontend linting configured
- Formatting strategy selected
- CI workflow running
- Local development checklist
- Pull request checklist

Files To Create Or Modify

- Create frontend/eslint.config.js or equivalent ESLint CLI configuration
- Create .github/workflows/ci.yml
- Create .editorconfig
- Create docs/release/pull-request-checklist.md
- Create docs/release/local-dev-checklist.md
- Update frontend/package.json
- Update README.md

Developer Steps

- Initialize ESLint using the modern CLI path rather than relying on deprecated next lint.
- Add a frontend lint script that runs non-interactively.
- Choose formatting policy:
 - Prettier, if you want standard frontend formatting fast
 - or ESLint-driven formatting boundaries if keeping tool count smaller
- Add CI jobs for:
 - backend tests
 - frontend build
 - frontend lint
- Ensure commands are deterministic and documented.
- Add failure messaging expectations so contributors know what broke and where.
- Add local pre-push discipline to README or release docs.

Quality Gates To Enforce

- No merge without passing backend tests.
- No merge without successful frontend build.
- No merge without frontend lint pass.
- New features should include tests if they add logic, permissions, or workflows.

Verification

- CI can run from a clean checkout.
- A frontend lint error fails the pipeline.
- A backend test failure fails the pipeline.
- Commands in the docs match the commands used by CI.

Exit Criteria

- The repo now has automated guardrails.
- Future work becomes safer and more reviewable.

Phase 5: Deliver The First Production-Style Vertical Slice

Objective: Ship one end-to-end collaboration flow that proves the project is more than a collection of features.

Recommended slice: Auth + profile + collaborative event workflow.

Why this slice first: It aligns best with the product's collaboration story and forces both frontend and backend to behave like a real multi-user system.

Developer workload focus: End-to-end execution, polish, acceptance criteria, and user-level correctness.

Scope Of This Slice

- Register user
- Log in user
- View/update profile
- Create event
- Invite participant
- Accept/decline invitation
- Display event correctly for creator and participant
- Reflect invitation status in UI
- Cover the flow with backend tests and at least basic frontend verification

Primary Files To Modify

- Modify frontend event components and dashboard pages created in Phase 2
- Modify backend planner models/serializers/views/services from Phase 3
- Update frontend/lib/api.js if API contract expands

Files To Create

- Create invitation-focused frontend UI components if not already introduced
- Create backend tests for invitation acceptance/decline API behavior
- Create docs/architecture/collaborative-event-slice.md

Developer Steps

- Finalize the API contract for invitations before polishing UI behavior.
- Add backend endpoints/actions for invite, accept, and decline.
- Verify event listing logic separately for:
 - creator
 - pending invitee
 - accepted participant
 - declined participant
- Build UI states for each invitation stage.

- Ensure event cards communicate role and status clearly.
- Ensure unauthorized actions are blocked both in UI and backend.
- Add regression coverage for old single-user event management flows.
- Review the full slice from the perspective of two separate accounts.

Acceptance Criteria

- User A can create an event and invite User B.
- User B can see the invitation and accept or decline it.
- User A sees the updated invitation status.
- Accepted events surface for both users according to the chosen visibility rules.
- Declined events stop behaving like active shared commitments.
- The UI makes roles and next actions obvious without explanation.

Verification

- Run full backend test suite.
- Run targeted collaboration tests.
- Run frontend production build.
- Manually test with two users.
- Confirm error states for duplicate invite, unauthorized update, and invalid status transition.

Exit Criteria

- The project now has one genuinely portfolio-grade workflow.
- Demo value increases because the app shows coordination, not just CRUD.

Phase 6: Challenge System Rework And Engagement Layer

Objective: Bring challenges and achievements up to the same quality bar as the planner collaboration slice.

Why after the event slice: The event slice establishes clearer collaboration foundations; challenge behavior should be rebuilt on top of that discipline instead of expanded prematurely.

Developer workload focus: Feature normalization, clearer rules, and engagement consistency.

Required Outputs

- Final challenge feature model
- Improved progress permissions
- More intentional achievement triggers
- Optional notification groundwork

Developer Steps

- Decide whether challenges remain personal in MVP and become group-based in V1, or whether minimal group challenge support is introduced now.

- Align challenge ownership and visibility rules with the product direction chosen in Phase 1.
- Review progress logging rules for abuse, duplication, and awkward edge cases.
- Rework achievement triggers around explicit domain events such as:
 - challenge completed
 - streak achieved
 - milestone count reached
- Decide which achievements are meaningful enough to keep visible in the product.
- Remove or postpone gamification that distracts from the core collaboration loop.

Verification

- Challenge logic feels coherent with the planner, not like a separate app inside the repo.
- Badge awarding remains idempotent and understandable.
- The UI communicates challenge state without needing internal knowledge.

Exit Criteria

- Challenges feel integrated into the product instead of appended.
- Engagement features reinforce the core user journey.

Phase 7: Deployment, Operations, And Portfolio Readiness

Objective: Make the project shippable, demonstrable, and maintainable beyond local development.

Why last: Deployment quality matters more once the product flow and engineering foundation are stable enough to preserve.

Developer workload focus: Environment strategy, deployment preparation, operations basics, and demonstration readiness.

Required Outputs

- Environment variable strategy
- Staging deployment
- PostgreSQL readiness path
- Media handling plan
- Error logging baseline
- Demo and release documentation

Files To Create Or Modify

- Create .env.example
- Create deployment notes in docs/release/deployment.md
- Create staging checklist in docs/release/staging-checklist.md
- Update backend settings strategy, potentially splitting settings modules
- Update README.md

Developer Steps

- Replace development-only assumptions in settings with environment-driven configuration.
- Decide how secrets are injected locally and in staging.
- Prepare PostgreSQL-compatible configuration even if SQLite remains local.
- Decide media handling path for deployed profile images.
- Add basic request/error logging.
- Add a lightweight health-check strategy.
- Validate that the app can be demonstrated without local hand-tuning.
- Prepare screenshots/demo script based on the first polished slice.

Verification

- Fresh environment setup works from the docs.
- Staging can be deployed from a clean branch.
- The main product demo does not require hidden setup tricks.
- Environment-specific failures are understandable and documented.

Exit Criteria

- The project is portfolio-grade in both code and presentation.
- You can explain architecture, workflow, and release approach like a real team project.

Recommended Weekly Operating Rhythm

This is how the project should be managed while we execute the plan.

- Start each week by choosing one phase checkpoint or one vertical-slice subgoal.
- Convert that checkpoint into a short implementation plan before coding.
- Implement in small reviewed commits.
- Run verification before claiming completion.
- End the week by documenting what changed, what remains risky, and what should happen next.

Priority Order Summary

1. Phase 1: Product Alignment And Delivery Strategy
2. Phase 2: Frontend Architecture Reset
3. Phase 3: Backend Collaboration Model Hardening
4. Phase 4: Engineering Quality Foundation
5. Phase 5: Deliver The First Production-Style Vertical Slice
6. Phase 6: Challenge System Rework And Engagement Layer
7. Phase 7: Deployment, Operations, And Portfolio Readiness

Suggested Effort Model

This is a rough workload framing, not a strict schedule.

- Phase 1: 1 to 2 focused sessions

- Phase 2: 4 to 7 development sessions
- Phase 3: 4 to 7 development sessions
- Phase 4: 2 to 4 development sessions
- Phase 5: 4 to 6 development sessions
- Phase 6: 3 to 5 development sessions
- Phase 7: 2 to 4 development sessions

The important point is sequence, not exact duration. We should finish one stable checkpoint before broadening scope again.

Self-Review

Spec Coverage

- Product direction covered
- Frontend work decomposition covered
- Backend collaboration hardening covered
- Quality automation covered
- Vertical slice covered
- Challenge and achievement evolution covered
- Deployment and operational readiness covered

Placeholder Scan

- No TODO
- No TBD
- No hidden "figure this out later" steps

Consistency Check

- Collaboration slice centers on events across frontend, backend, tests, and deployment narrative
- Challenges are intentionally positioned after collaboration hardening, not mixed into the first critical slice
- Phase order protects maintainability before feature expansion

Execution Handoff

Plan complete and saved to docs/superpowers/plans/2026-05-29-planquest-master-execution-plan.md.

Two execution options:

8. Subagent-Driven (recommended) - implement one checkpoint at a time with review between tasks
9. Inline Execution - execute the chosen phase directly in this session